LAB ASSIGNMENT-13.4

Name: N.Soumya

H NO:2303A51869

Batch-13

Task 1: Refactoring Data Transformation Logic

Scenario

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Task Description

• Review the legacy code that computes transformed values using an explicit loop.

• Use an AI tool to suggest a more Pythonic refactoring approach.

• Refactor the code using list comprehensions or helper functions while preserving the output.

Legacy Code

```
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
doubled.append(v * 2)
print(doubled)
```

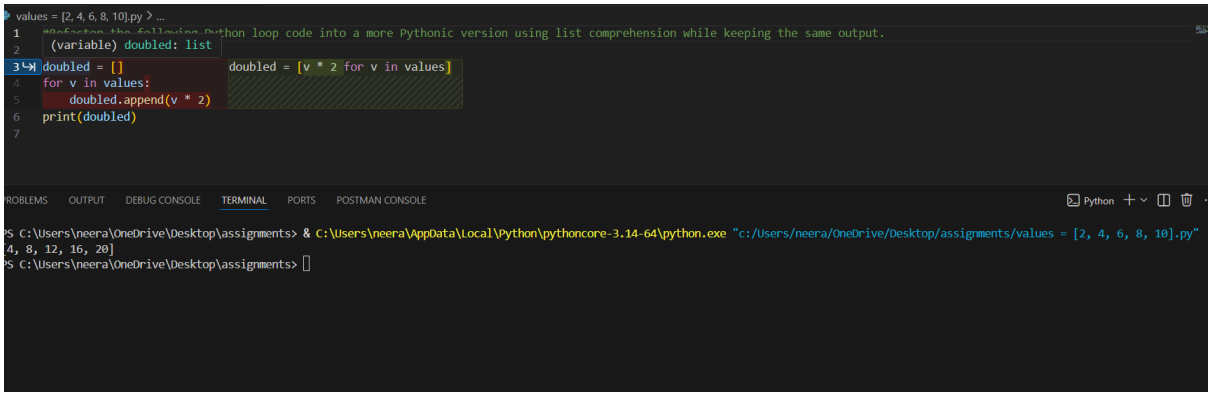
Expected Output

[4, 8, 12, 16, 20]

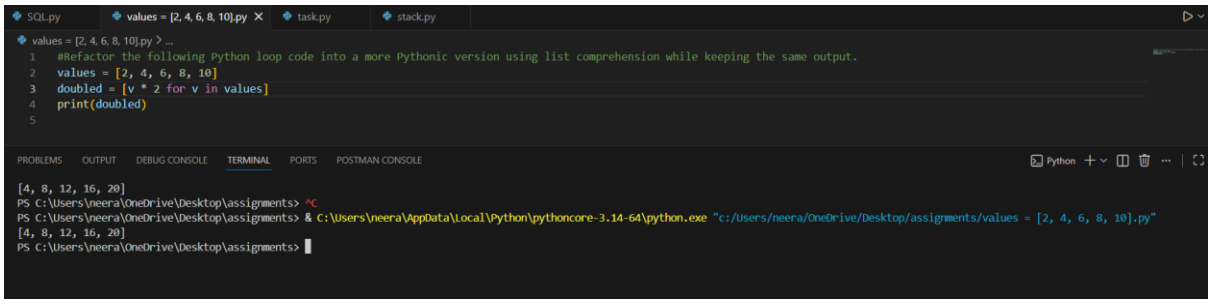
#PROMPT:

#Refactor the following Python loop code into a more Pythonic version using list comprehension while keeping the same output

#Original Code



#Refactored Code



#Justification

The original code uses a traditional for loop to iterate over the list of values and append the doubled values to a new list. The refactored version uses a list comprehension, which is more concise and often considered more readable in Python. List comprehensions allow you to create a new list by applying an expression to each item in an iterable, all in one line.

Task 2: Improving Text Processing Code Readability

Scenario

You are working on a log message generator that builds messages word by word.

Task Description

- Analyze the legacy code that constructs a sentence using repeated string concatenation.
- Ask AI to suggest a more efficient and readable approach.

• Refactor the code accordingly while keeping the final output unchanged.

Legacy Code

words = ["Refactoring", "with", "AI", "improves", "quality"]

message = ""

for w in words:

message += w + " "

print(message.strip())

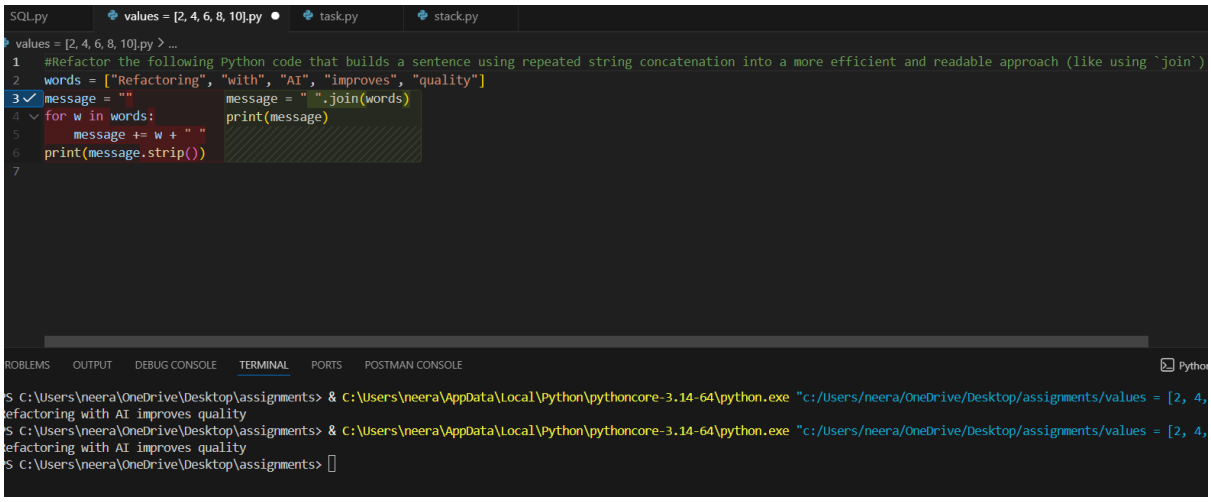
Expected Output

Refactoring with AI improves quality

#PROMPT:

Refactor the following Python code that builds a sentence using repeated string concatenation into a more efficient and readable approach .

#Original Code



#Refactored Code

#Justification

Using “join” is more efficient than repeated string concatenation because it creates the final string in one step, rather than creating multiple intermediate strings. This approach is not only faster but also more readable, as it clearly shows that we are combining a list of words into a single sentence.

Task 3: Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may

or may not exist depending on deployment settings.

Task Description

• Review the legacy code that manually checks for dictionary keys.

• Use AI suggestions to refactor the code using safer dictionary

access methods.

• Ensure the behavior remains the same for missing keys.

Legacy Code

```
config = {"host": "localhost", "port": 8080}

if "timeout" in config:

print(config["timeout"])

else:

print("Default timeout used")
```

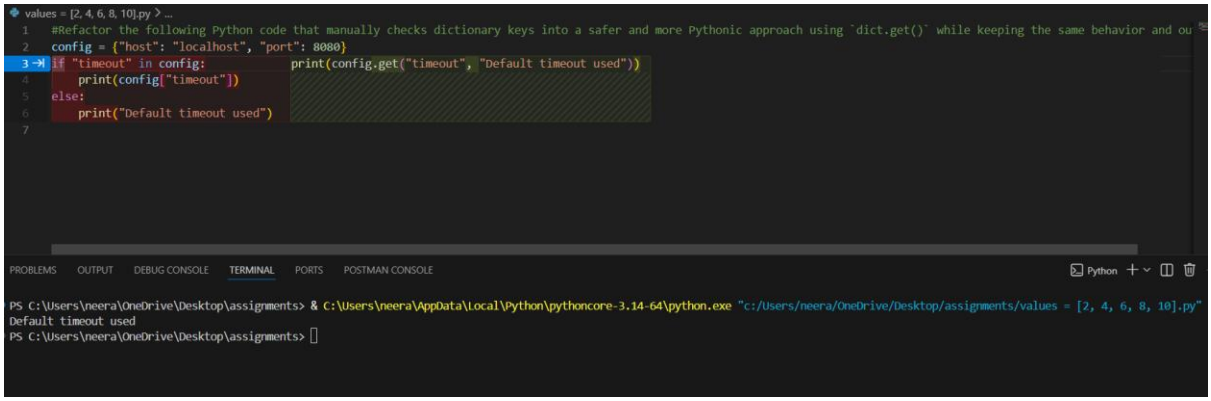
Expected Output

Default timeout used

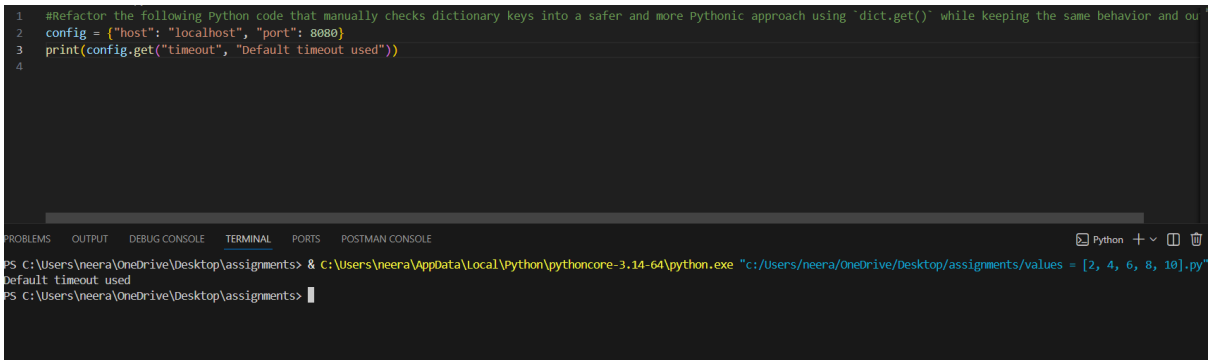
#PROMPT

Refactor the following Python code that manually checks dictionary keys into a safer and more Pythonic approach using dict.get() .

#Original Code



#Refactored Code



#Justification

The original code checks if the key exists before accessing it, which makes the code longer.Using dict.get() simplifies the process by safely accessing the value.If the key is missing, it can return a default value instead of causing an error.This makes the code cleaner, safer, and more Pythonic.

Task 4: Refactoring Conditional Logic for Scalability

Scenario

You are enhancing a utility module that performs different operations

based on user input.

Task Description

- Examine the multiple if–elif conditions used to determine operations.
- Ask AI to suggest a cleaner, scalable alternative.
- Refactor the logic using mapping techniques while preserving

functionality.

Legacy Code

action = "divide"

x, y = 10, 2

if action == "add":

result = x + y

elif action == "subtract":

result = x - y

elif action == "multiply":

result = x * y

elif action == "divide":

result = x / y

else:

result = None

print(result)

Expected Output

5.0

#PROMPT

Refactor the following Python code that uses multiple if–elif conditions into a cleaner and scalable solution using a dictionary mapping

#Original Code

```
#Refactor the following Python code that uses multiple if-elif conditions into a cleaner and scalable solution using a dictionary mapping
action = "divide"
x, y = 10, 2
if action == "add":
    result = x + y
elif action == "subtract":
    result = x - y
elif action == "multiply":
    result = x * y
elif action == "divide":
    result = x / y
else:
    result = None
print(result)
```

```
x, y = 10, 2

operations = {
    "add": lambda a, b: a + b,
    "subtract": lambda a, b: a - b,
    "multiply": lambda a, b: a * b,
    "divide": lambda a, b: a / b
}

result = operations.get(action, lambda a, b: None)(x, y)
print(result)
```

#Refactored Code

```python
#Refactor the following Python code that uses multiple if-elif conditions into a cleaner and scalable solution using a dictionary mapping
action = "divide"
x, y = 10, 2

operations = {
    "add": lambda a, b: a + b,
    "subtract": lambda a, b: a - b,
    "multiply": lambda a, b: a * b,
    "divide": lambda a, b: a / b
}
result = operations.get(action, lambda a, b: None)(x, y)
print(result)
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE                    Python

PS C:\Users\neera\OneDrive\Desktop\assignments> & C:\Users\neera\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/neera/OneDrive/Desktop/assignments/values = [2, 4, 6, 8, 10].py"
5.0
PS C:\Users\neera\OneDrive\Desktop\assignments>
```

#Justification

This refactored code uses a dictionary to map string keys (the actions) to their corresponding operations (implemented as lambda functions). This approach eliminates the need for multiple if–elif conditions, making the code cleaner and more scalable.

Task 5: Simplifying Search Logic in Collections

Scenario

You are reviewing legacy inventory-check code that uses manual loops to find elements.

Task Description

• Identify the explicit loop used for searching an item.

• Use AI assistance to refactor the logic into a more concise and

readable form.

• Maintain the same output behavior.

Legacy Code

inventory = ["pen", "notebook", "eraser", "marker"]

found = False

for item in inventory:

if item == "eraser":

found = True

break

print("Item Available" if found else "Item Not Available")

Expected Output

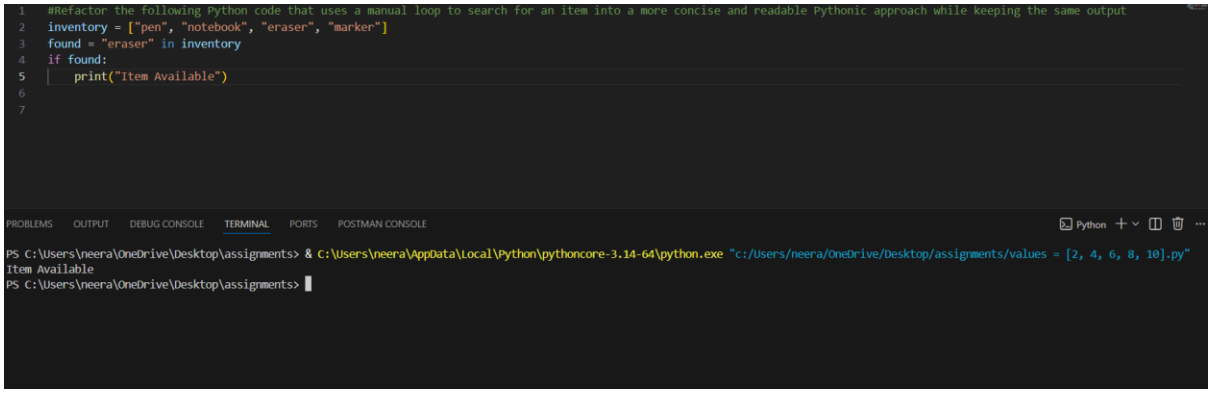
Item Available

#PROMPT

#Refactor the following Python code that uses a manual loop to search for an item into a more concise and readable Pythonic approach while keeping the same output

#Original Code

#Refactored Code



#Justification

The refactored code uses the “in” keyword, which is a more concise and readable way to check for the presence of an item in a list. This approach eliminates the need for a manual loop and a separate variable to track whether the item was found, making the code cleaner and easier to understand.